Solving Generalized Sudoku with SAT, CP-SAT and Gurobi MIP

Nguyen Tran Hoang Hieu
VNU University of Engineering and Technology,

1. Introduction

This project extends a 9×9 Sudoku solver to $N \times N$ boards and compares three approaches: SAT with PySAT/Glucose3, OR-Tools CP-SAT, and Gurobi MIP. It provides separate puzzle files, correctness tests, and reproducible experiments. The SAT model follows established Sudoku encodings [1] and uses sequential counters for cardinality constraints [2]. The comparison covers a small synthetic dataset; CPLEX CP and CPLEX MIP are not included.

2. Problem definition

Let $N = k^2$, where k is a positive integer. The input is an $N \times N$ matrix P with values in $\{0, \dots, N\}$; zero denotes an empty cell. A solution assigns values $1, \dots, N$ so that every row, column, and $k \times k$ block contains each value exactly once and preserves all clues. The program returns one solution or reports unsatisfiability. Invalid inputs are rejected separately, and solution uniqueness is not checked.

3. Solving methods

3.1. SAT

A Boolean variable $x_{r,c,v}$ represents value v in cell (r, c), with identifier $N^2r + Nc + v$ for zero-based r, c and $1 \le v \le N$. Exactly-one constraints enforce one value per cell and one occurrence of each value per row, column, and block. Each clue adds a unit clause $(x_{r,c,v})$.

Each exactly-one constraint combines an at-least-one clause with a sequential-counter at-most-one encoding [2]. For inputs $z_1, \dots, z_N$, auxiliary variables $s_1, \dots, s_{N-1}$ form the clauses

$$(\neg z_1 \vee s_1), \quad (\neg z_N \vee \neg s_{N-1}), \tag{1}$$

$$(\neg z_i \vee s_i), \quad (\neg s_{i-1} \vee s_i), \quad (\neg z_i \vee \neg s_{i-1}) \quad (2 \le i < N). \tag{2}$$

The chain prevents two inputs from being true. Together with the clue clauses, the encoding is satisfiable exactly when the puzzle has a valid completion.

For $N \ge 2$ and G clues, the formula contains

$$N^3 + 4N^2(N-1) \text{ variables}, \qquad 4N^2(3N-3) + G \text{ clauses}.$$

The $N = 1$ case needs no auxiliary variables. Clauses are streamed to Glucose3 through PySAT [3], and true cell-value variables are decoded into the solution.

3.2. CP-SAT

Each cell is an integer variable $Y_{r,c} \in \{1, \dots, N\}$. The model imposes $3N$ all-different constraints over rows, columns, and blocks, plus an equality for each clue. CP-SAT combines SAT reasoning with constraint and integer programming techniques [4]. Experiments use one worker.

3.3. Gurobi MIP

Binary variables $z_{r,c,v}$ represent cell-value assignments. The four constraint families used by SAT become linear equations $\sum z = 1$, while clues impose $z_{r,c,v} = 1$. The model has N^3 binary variables and $4N^2 + G$ equations. Gurobi [5] uses one thread and stops after the first solution.

Both comparison models solve a feasibility problem. A valid solution means SAT, a proven infeasible model means UNSAT, and a limit reached without a solution remains UNKNOWN. An OPTIMAL status does not establish uniqueness.

Table 1
Median model plus solve time in seconds over three repetitions. SAT denotes Glucose3; Gurobi uses a binary MIP; NR denotes not run.

Instance	N	Blanks	Status	SAT (s)	CP-SAT (s)	Gurobi (s)
04_blank_8	4	8	SAT	0.0006	0.0052	0.0097
04_blank_12	4	12	SAT	0.0005	0.0050	0.0110
04_blank_16	4	16	SAT	0.0004	0.0107	0.0096
09_blank_40	9	40	SAT	0.0050	0.0071	0.0159
09_blank_60	9	60	SAT	0.0049	0.0203	0.0206
16_blank_128	16	128	SAT	0.0301	0.0151	0.0436
16_blank_192	16	192	SAT	0.0314	0.1010	0.0594
25_blank_200	25	200	SAT	0.1285	0.0163	0.1245
25_blank_400	25	400	SAT	0.1877	0.2440	10.3765
36_blank_360	36	360	SAT	0.4185	0.1382	0.4317
49_blank_490	49	490	SAT	1.0604	0.9954	1.5177
100_blank_1	100	1	SAT	NR	0.1836	NR
04_unsat_row	4	14	UNSAT	0.0006	0.0134	0.0133
04_unsat_column	4	14	UNSAT	0.0006	0.0037	0.0173
04_unsat_box	4	14	UNSAT	0.0005	0.0225	0.0101
04_unsat_no_duplicate	4	10	UNSAT	0.0004	0.0039	0.0101

4. Experimental study

4.1. Dataset and validation

The dataset contains synthetic satisfiable boards of sizes 4, 9, 16, 25, 36, 49, and 100, generated by deleting cells from deterministic completed grids with fixed random seeds. The original grids serve as solution witnesses. Four unsatisfiable 4×4 puzzles include three direct clue conflicts and one case independently verified against all 288 completed 4×4 grids.

Every returned solution is checked against the clues and Sudoku rules. Invalid inputs are tested separately. All three methods are compared through 49×49. The 100×100 case has one empty cell and is run only with CP-SAT; SAT and Gurobi are unmeasured on this case.

4.2. Setup and timing

The recorded environment is Windows 11, an AMD Ryzen 7 8845H, Python 3.12.14, python-sat 1.9.dev16, OR-Tools 9.15.6755, and Gurobi 13.0.3. Each case is repeated three times in fresh processes, with method order reversed on alternating repetitions and a 30-second external limit per process.

Reported time is model construction plus solving, excluding process startup, file loading, decoding, and validation. SAT construction includes clause insertion; CP-SAT presolve occurs during solving; Gurobi construction includes environment startup and license validation. These boundaries and uncontrolled system load affect comparisons.

4.3. Results

All 138 runs across 16 instances returned the expected status, and every returned solution passed validation. Table 1 gives median times. For the 25×25 puzzle with 400 blanks, SAT, CP-SAT, and Gurobi took 0.1877 s, 0.2440 s, and 10.3765 s, respectively. For 49×49, the medians were 1.0604 s, 0.9954 s, and 1.5177 s; CP-SAT runs ranged from 0.0701 s to 1.2508 s, showing substantial variation.

CP-SAT solved the nearly complete 100×100 puzzle in a median 0.1836 s. At this size, the SAT input would contain 4.96 million variables and approximately 11.89 million clauses; Gurobi would use one million binary variables. CP-SAT exposes 10,000 integer variables and 300 all-different constraints, plus clues. These are input-model counts, not measurements of internal memory use.

4.4. Limitations

The dataset shares one regular construction and includes few large puzzles, some almost complete. Results therefore describe these instances rather than establish a general solver ranking. Peak memory, uniqueness, alternative AMO encodings, independent UNSAT proof checking, and harder published benchmarks remain untested.

5. Conclusion

The project implements and validates three approaches to generalized Sudoku, with comparisons through 49×49 and a separate nearly complete 100×100 CP-SAT example. Performance varies by puzzle and formulation. Future work should expand the benchmark set, measure peak memory, compare AMO encodings, and add CPLEX baselines.

Reproducibility

Repository: `https://github.com/YoungMilos/Sudoku_pySAT_solver`. The report directory contains raw results, environment metadata, and scripts. After installing `requirements.txt` and configuring a Gurobi license, run `reports/sudoku_cp_sat_report/benchmark.py` from the project root, then `reports/sudoku_cp_sat_report/make_table.py`. These scripts overwrite measurements and the table; update the numerical discussion before recompiling. Commit and push the report files before submission.

Author and course information

Nguyen Tran Hoang Hieu; student ID 24021477; class 2627I_INT3011E_3; supervisor To Van Khanh.

References

[1] I. Lynce and J. Ouaknine, Sudoku as a SAT Problem, 9th International Symposium on Artificial Intelligence and Mathematics, 2006. https://people.mpi-sws.org/~joel/publications/sudoku05abs.html.

[2] C. Sinz, Towards an Optimal CNF Encoding of Boolean Cardinality Constraints, CP 2005, pp. 827–831, 2005. doi:10.1007/11564751_73.

[3] A. Ignatiev, A. Morgado, and J. Marques-Silva, PySAT: A Python Toolkit for Prototyping with SAT Oracles, SAT 2018, pp. 428–437, 2018. doi:10.1007/978-3-319-94144-8_26.

[4] L. Perron, F. Didier, and S. Gay, The CP-SAT-LP Solver, CP 2023. doi:10.4230/LIPIcs.CP.2023.3.

[5] Gurobi Optimization, LLC, Gurobi Optimizer Reference Manual, version 13.0, 2026. https://docs.gurobi.com/projects/optimizer/en/current/.